

Window Function Interview Problems: 20+ Company-Style Tasks with Solutions

Table of Contents

1. [Learning Objectives](#)
 2. [How to Approach Window Function Problems](#)
 3. [Setup: Schema and Data](#)
 4. [EASY Problems \(P1-P5\)](#)
 5. [MEDIUM Problems \(P6-P12\)](#)
 6. [HARD Problems \(P13-P18\)](#)
 7. [EXPERT Problems \(P19-P22\)](#)
 8. [ASCII Visual Diagrams](#)
 9. [Common Interview Mistakes](#)
 10. [Interview Strategy Guide](#)
 11. [Best Practices for Interviews](#)
 12. [Performance Considerations](#)
 13. [Interview Questions & Answers](#)
 14. [Additional Practice Exercises](#)
 15. [Advanced Notes](#)
 16. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Solve 20+ real company-style window function interview problems
 - Recognize common patterns (top-N, running totals, gaps, streaks)
 - Translate verbal problem descriptions into window function queries
 - Explain your approach clearly in technical interviews
 - Handle edge cases (NULLs, ties, empty groups) correctly
-

How to Approach Window Function Problems

STEP 1: Understand the output

- How many rows should the result have?
- Which columns from which tables?
- Any aggregation or ranking needed?

STEP 2: Identify the pattern

- Ranking → ROW_NUMBER / RANK / DENSE_RANK
- Running total → SUM/AVG OVER (ORDER BY)
- Period comparison → LAG / LEAD
- First/Last in group → FIRST_VALUE / LAST_VALUE
- N-th value → NTH_VALUE
- Bucket assignment → NTILE
- Deduplication → ROW_NUMBER

STEP 3: Define the window

- PARTITION BY: what is the "group"?
- ORDER BY: what determines position within the group?
- Frame: ROWS BETWEEN / RANGE BETWEEN needed?

STEP 4: Write incrementally

- Start with the inner/window query
- Wrap in CTE or subquery to filter/transform
- Test with a small result set

STEP 5: Handle edge cases

- NULLs in ORDER BY or PARTITION BY
- Ties in ORDER BY → RANK vs ROW_NUMBER
- First/last rows with no predecessor/successor

Setup

```
-- Products and sales tables
CREATE TABLE products_iv (
  product_id  INT PRIMARY KEY,
  product_name VARCHAR(50),
  category    VARCHAR(30),
  price       NUMERIC(10,2)
);

CREATE TABLE sales_iv (
  sale_id      SERIAL PRIMARY KEY,
  product_id   INT,
  customer_id  INT,
  sale_date    DATE,
  quantity     INT,
  amount       NUMERIC(10,2),
  region       VARCHAR(20)
);

CREATE TABLE employees_iv (
  emp_id       INT PRIMARY KEY,
  emp_name     VARCHAR(50),
  department   VARCHAR(30),
  salary       NUMERIC(10,2),
  hire_date    DATE,
  manager_id   INT
);

CREATE TABLE user_activity (
  user_id      INT,
  activity_date DATE,
  page_views   INT,
  purchases    INT
```

```
);
```

```
INSERT INTO products_iv VALUES
```

```
(1, 'Laptop', 'Electronics', 1200),
(2, 'Phone', 'Electronics', 800),
(3, 'Tablet', 'Electronics', 600),
(4, 'Chair', 'Furniture', 300),
(5, 'Desk', 'Furniture', 500),
(6, 'Headphones', 'Electronics', 150),
(7, 'Keyboard', 'Electronics', 80),
(8, 'Monitor', 'Electronics', 400);
```

```
INSERT INTO employees_iv VALUES
```

```
(1, 'Alice', 'Engineering', 120000, '2018-03-01', NULL),
(2, 'Bob', 'Engineering', 95000, '2019-06-15', 1),
(3, 'Carol', 'Engineering', 87000, '2020-01-10', 1),
(4, 'David', 'Marketing', 75000, '2019-09-01', NULL),
(5, 'Eve', 'Marketing', 68000, '2021-03-20', 4),
(6, 'Frank', 'Marketing', 72000, '2020-07-01', 4),
(7, 'Grace', 'Sales', 82000, '2018-11-15', NULL),
(8, 'Henry', 'Sales', 65000, '2022-01-05', 7),
(9, 'Iris', 'Sales', 78000, '2019-04-22', 7),
(10, 'Jack', 'Sales', 61000, '2023-03-10', 7);
```

```
INSERT INTO sales_iv (product_id, customer_id, sale_date, quantity, amount, region)
VALUES
```

```
(1, 101, '2024-01-05', 2, 2400, 'North'),
(2, 102, '2024-01-10', 1, 800, 'South'),
(3, 103, '2024-01-15', 3, 1800, 'East'),
(1, 101, '2024-02-01', 1, 1200, 'North'),
(4, 104, '2024-02-10', 4, 1200, 'West'),
(2, 105, '2024-02-15', 2, 1600, 'South'),
(5, 102, '2024-03-01', 1, 500, 'South'),
(6, 103, '2024-03-10', 5, 750, 'East'),
(1, 106, '2024-03-15', 1, 1200, 'North'),
(7, 101, '2024-04-01', 3, 240, 'North'),
(3, 107, '2024-04-10', 2, 1200, 'West'),
(8, 104, '2024-04-20', 1, 400, 'West'),
(2, 105, '2024-05-01', 1, 800, 'South'),
(6, 106, '2024-05-15', 3, 450, 'North'),
(1, 107, '2024-05-20', 2, 2400, 'East');
```

```
INSERT INTO user_activity VALUES
```

```
(1, '2024-01-01', 10, 2), (1, '2024-01-02', 15, 0),
(1, '2024-01-03', 8, 1), (1, '2024-01-04', 12, 3),
(2, '2024-01-01', 5, 0), (2, '2024-01-02', 20, 1),
(2, '2024-01-03', 0, 0), (2, '2024-01-04', 8, 2),
(1, '2024-01-07', 18, 1),
(2, '2024-01-06', 12, 0);
```

EASY Problems

P1: Rank Employees by Salary Within Department

Problem: For each employee, show their salary rank within their department. Employees with the same salary should share the same rank. No gaps in ranking.

```
-- Solution:
SELECT
    emp_name,
    department,
    salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees_iv
ORDER BY department, dept_rank;

-- Key insight: DENSE_RANK (no gaps) vs RANK (gaps after ties)
-- "No gaps" → always DENSE_RANK
```

P2: Running Total of Sales

Problem: For each sale, show a running total of the amount, ordered by sale date.

```
-- Solution:
SELECT
    sale_id,
    sale_date,
    amount,
    SUM(amount) OVER (ORDER BY sale_date, sale_id
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                      ) AS running_total
FROM sales_iv
ORDER BY sale_date, sale_id;

-- Key insight: Use ROWS not RANGE to avoid aggregating ties
-- Use sale_id as tie-breaker for determinism
```

P3: Compare Salary to Department Average

Problem: Show each employee's name, salary, their department average, and the difference.

```
-- Solution:
SELECT
    emp_name,
    department,
    salary,
    ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
    salary - ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS diff
FROM employees_iv
ORDER BY department, salary DESC;
```

P4: Assign Sales to Quartiles

Problem: Divide all sales into 4 buckets by amount (highest amount = bucket 1).

```
-- Solution:
SELECT
    sale_id,
    sale_date,
    amount,
    NTILE(4) OVER (ORDER BY amount DESC) AS quartile,
    CASE NTILE(4) OVER (ORDER BY amount DESC)
        WHEN 1 THEN 'Top 25%'
        WHEN 2 THEN '25-50%'
        WHEN 3 THEN '50-75%'
        WHEN 4 THEN 'Bottom 25%'
    END AS bucket_label
FROM sales_iv
ORDER BY amount DESC;
```

P5: Previous Month Revenue Comparison

Problem: For each product's monthly sales, show the previous month's revenue and the change.

```
-- Solution:
SELECT
    product_id,
    DATE_TRUNC('month', sale_date) AS month,
    SUM(amount) AS revenue,
    LAG(SUM(amount)) OVER (PARTITION BY product_id ORDER BY DATE_TRUNC('month',
sale_date)) AS prev_revenue,
    SUM(amount) - LAG(SUM(amount)) OVER (PARTITION BY product_id ORDER BY
DATE_TRUNC('month', sale_date)) AS change
FROM sales_iv
GROUP BY product_id, DATE_TRUNC('month', sale_date)
ORDER BY product_id, month;
```

MEDIUM Problems

P6: Top-2 Employees by Salary per Department

Problem: Find the top-2 highest-paid employees in each department. If there are ties, include all tied employees.

```
-- Solution:
SELECT emp_name, department, salary, dept_rank
FROM (
    SELECT
        emp_name,
        department,
        salary,
```

```

        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS
dept_rank
    FROM employees_iv
) AS ranked
WHERE dept_rank <= 2
ORDER BY department, dept_rank;

-- Key insight: Use DENSE_RANK so ties are both included
-- Use ROW_NUMBER if you strictly want exactly 2 rows (no ties)

```

P7: Month-over-Month Growth Rate

Problem: For each region, compute monthly revenue and month-over-month growth percentage.

```

-- Solution:
WITH monthly AS (
    SELECT
        region,
        DATE_TRUNC('month', sale_date) AS month,
        SUM(amount) AS revenue
    FROM sales_iv
    GROUP BY region, DATE_TRUNC('month', sale_date)
)
SELECT
    region,
    month,
    revenue,
    LAG(revenue) OVER (PARTITION BY region ORDER BY month) AS prev_revenue,
    ROUND(100.0 * (revenue - LAG(revenue) OVER (PARTITION BY region ORDER BY month))
        / NULLIF(LAG(revenue) OVER (PARTITION BY region ORDER BY month), 0), 2
    ) AS mom_growth_pct
FROM monthly
ORDER BY region, month;

```

P8: Cumulative Distribution of Salaries

Problem: For each employee, show what percentage of all employees earn less than or equal to them.

```

-- Solution:
SELECT
    emp_name,
    department,
    salary,
    ROUND(CUME_DIST() OVER (ORDER BY salary) * 100, 2) AS salary_percentile,
    ROUND(PERCENT_RANK() OVER (ORDER BY salary) * 100, 2) AS salary_percent_rank
FROM employees_iv
ORDER BY salary DESC;

-- Insight: CUME_DIST = fraction of rows with salary <= current
-- PERCENT_RANK = (rank-1)/(total-1), ranges from 0 to 1

```

P9: 3-Month Rolling Average Revenue

Problem: For each region, compute a 3-month rolling average of revenue.

```
-- Solution:
WITH monthly AS (
    SELECT
        region,
        DATE_TRUNC('month', sale_date) AS month,
        SUM(amount) AS revenue
    FROM sales_iv
    GROUP BY region, DATE_TRUNC('month', sale_date)
)
SELECT
    region,
    month,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY region
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS rolling_avg_3m,
    COUNT(*) OVER (
        PARTITION BY region
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS window_size
FROM monthly
ORDER BY region, month;
```

P10: Find the Nth Highest Salary in Each Department

Problem: Find employees with the 3rd highest salary in each department using window functions.

```
-- Solution:
SELECT emp_name, department, salary
FROM (
    SELECT
        emp_name,
        department,
        salary,
        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
    FROM employees_iv
) AS ranked
WHERE dr = 3
ORDER BY department;

-- If no 3rd distinct salary exists in a department, that dept returns no rows
```

P11: First and Last Sale per Customer

Problem: For each customer, show their first and most recent purchase date and amounts.

```
-- Solution:
SELECT DISTINCT
    customer_id,
    FIRST_VALUE(sale_date) OVER w_asc AS first_sale_date,
    FIRST_VALUE(amount) OVER w_asc AS first_sale_amount,
    LAST_VALUE(sale_date) OVER w_full AS last_sale_date,
    LAST_VALUE(amount) OVER w_full AS last_sale_amount,
    COUNT(*) OVER (PARTITION BY customer_id) AS total_orders,
    SUM(amount) OVER (PARTITION BY customer_id) AS lifetime_value
FROM sales_iv
WINDOW
    w_asc AS (PARTITION BY customer_id ORDER BY sale_date
              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING),
    w_full AS (PARTITION BY customer_id ORDER BY sale_date
              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
ORDER BY customer_id;
```

P12: Employees Earning More Than Department Average

Problem: List all employees who earn more than the average salary of their department, showing the department average alongside.

```
-- Solution:
SELECT emp_name, department, salary, ROUND(dept_avg, 2) AS dept_avg
FROM (
    SELECT
        emp_name,
        department,
        salary,
        AVG(salary) OVER (PARTITION BY department) AS dept_avg
    FROM employees_iv
) AS t
WHERE salary > dept_avg
ORDER BY department, salary DESC;
```

HARD Problems

P13: Deduplication — Keep Latest Record per Customer-Product

Problem: The sales table has duplicate entries (same customer, product, date). Keep only the most recent entry (highest sale_id) per customer-product combination.

```
-- Solution:
WITH ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id, product_id
```

```

        ORDER BY sale_date DESC, sale_id DESC
    ) AS rn
FROM sales_iv
)
SELECT sale_id, product_id, customer_id, sale_date, amount
FROM ranked
WHERE rn = 1
ORDER BY customer_id, product_id;

```

P14: Gap and Island Detection (Consecutive Login Days)

Problem: Find users with consecutive activity streaks (consecutive days). Return the start and end date of each streak and its length.

```

-- Solution:
WITH date_groups AS (
    SELECT
        user_id,
        activity_date,
        activity_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
activity_date))::INT
        AS grp
    FROM user_activity
),
streaks AS (
    SELECT
        user_id,
        MIN(activity_date) AS streak_start,
        MAX(activity_date) AS streak_end,
        COUNT(*)          AS streak_length,
        grp
    FROM date_groups
    GROUP BY user_id, grp
)
SELECT user_id, streak_start, streak_end, streak_length
FROM streaks
ORDER BY user_id, streak_start;

-- Technique: consecutive dates minus row_number = constant for a streak
-- Non-consecutive gaps produce different group values

```

P15: Highest Product Revenue per Category with Running Total

Problem: For each product, show its total revenue, rank within its category, and a running total within the category ordered by revenue descending.

```

-- Solution:
WITH product_revenue AS (
    SELECT
        p.product_id,
        p.product_name,

```

```

        p.category,
        SUM(s.amount) AS total_revenue
FROM products_iv p
JOIN sales_iv s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name, p.category
)
SELECT
    product_name,
    category,
    total_revenue,
    RANK() OVER (PARTITION BY category ORDER BY total_revenue DESC) AS
category_rank,
    SUM(total_revenue) OVER (
        PARTITION BY category
        ORDER BY total_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total_in_category,
    ROUND(100.0 * total_revenue /
        SUM(total_revenue) OVER (PARTITION BY category), 2) AS pct_of_category
FROM product_revenue
ORDER BY category, category_rank;

```

P16: Year-over-Year Growth (Facebook/Stripe style)

Problem: For each region, compare Q1 2024 revenue to Q1 2023 revenue and show YoY growth. (Simulate with available data.)

```

-- Solution:
WITH quarterly AS (
    SELECT
        region,
        EXTRACT(YEAR FROM sale_date) AS yr,
        EXTRACT(QUARTER FROM sale_date) AS qtr,
        SUM(amount) AS revenue
    FROM sales_iv
    GROUP BY region, yr, qtr
),
with_prev_year AS (
    SELECT
        region,
        yr,
        qtr,
        revenue,
        LAG(revenue) OVER (PARTITION BY region, qtr ORDER BY yr) AS
prev_year_revenue
    FROM quarterly
)
SELECT
    region,
    yr,
    qtr,

```

```

    revenue,
    prev_year_revenue,
    ROUND(100.0 * (revenue - prev_year_revenue) / NULLIF(prev_year_revenue, 0), 2)
AS yoy_pct
FROM with_prev_year
WHERE prev_year_revenue IS NOT NULL
ORDER BY region, yr, qtr;

```

P17: Running Maximum (High Water Mark)

Problem: For each region, show the running maximum sale amount as of each sale date (the "record high" at any point in time).

```

-- Solution:
SELECT
    region,
    sale_date,
    amount,
    MAX(amount) OVER (
        PARTITION BY region
        ORDER BY sale_date, sale_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_max,
    CASE
        WHEN amount = MAX(amount) OVER (
            PARTITION BY region
            ORDER BY sale_date, sale_id
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) THEN 'NEW RECORD!'
        ELSE ''
    END AS is_record
FROM sales_iv
ORDER BY region, sale_date;

```

P18: Median Salary per Department (without PERCENTILE_CONT)

Problem: Compute the median salary for each department using window functions (for systems without PERCENTILE_CONT).

```

-- Solution using ROW_NUMBER and COUNT:
WITH ranked AS (
    SELECT
        emp_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary) AS rn_asc,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn_desc,
        COUNT(*) OVER (PARTITION BY department) AS dept_count
    FROM employees_iv
)
SELECT

```

```

    department,
    AVG(salary) AS median_salary    -- average of middle row(s)
FROM ranked
WHERE rn_asc IN (dept_count / 2, dept_count / 2 + 1)    -- handle even counts
    OR (dept_count % 2 = 1 AND rn_asc = (dept_count + 1) / 2)
GROUP BY department
ORDER BY department;

-- Cleaner alternative with PERCENTILE_CONT:
SELECT department,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median
FROM employees_iv GROUP BY department;

```

EXPERT Problems

P19: Self-Referencing Window — Percentage of Running Total

Problem: Show each sale's percentage contribution to the running total of its region (not the final total — the percentage of the total accumulated so far at each point).

```

-- Solution:
SELECT
    region,
    sale_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY region ORDER BY sale_date, sale_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total,
    ROUND(100.0 * amount /
        SUM(amount) OVER (
            PARTITION BY region ORDER BY sale_date, sale_id
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ), 2) AS pct_of_running_total
FROM sales_iv
ORDER BY region, sale_date;

```

P20: Salary Bands with Running Count

Problem: Divide employees into salary bands (every \$20K), count employees per band, and show cumulative headcount.

```

-- Solution:
WITH banded AS (
    SELECT
        emp_name,
        salary,
        (FLOOR(salary / 20000) * 20000)::INT AS band_start,
        (FLOOR(salary / 20000) * 20000 + 19999)::INT AS band_end
    FROM employees_iv

```

```

),
counts AS (
    SELECT band_start, band_end, COUNT(*) AS band_count
    FROM banded
    GROUP BY band_start, band_end
)
SELECT
    band_start,
    band_end,
    band_count,
    SUM(band_count) OVER (ORDER BY band_start
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_headcount,
    ROUND(100.0 * band_count / SUM(band_count) OVER (), 2) AS pct_of_total
FROM counts
ORDER BY band_start;

```

P21: Multi-Level Running Total (Amazon/Google style)

Problem: For each sale, show: the running total for that product, the running total for that region, and the global running total — all in one query.

```

-- Solution:
SELECT
    sale_id,
    sale_date,
    product_id,
    region,
    amount,
    SUM(amount) OVER (
        PARTITION BY product_id
        ORDER BY sale_date, sale_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS product_running_total,
    SUM(amount) OVER (
        PARTITION BY region
        ORDER BY sale_date, sale_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS region_running_total,
    SUM(amount) OVER (
        ORDER BY sale_date, sale_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS global_running_total
FROM sales_iv
ORDER BY sale_date, sale_id;

```

P22: Active Users Calculation (Stripe/Meta style)

Problem: For each day, count the number of users who were active on that day AND on at least one of the 2 previous days (rolling 3-day active users).

```
-- Solution:
WITH daily_users AS (
    SELECT DISTINCT user_id, activity_date FROM user_activity
),
windowed AS (
    SELECT
        user_id,
        activity_date,
        COUNT(*) OVER (
            PARTITION BY user_id
            ORDER BY activity_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ) AS days_active_in_3d_window,
        MIN(activity_date) OVER (
            PARTITION BY user_id
            ORDER BY activity_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ) AS window_start
    FROM daily_users
)
SELECT
    activity_date,
    COUNT(DISTINCT user_id) AS rolling_3d_active_users
FROM windowed
WHERE days_active_in_3d_window >= 2 -- active today + at least 1 of prior 2 days
    AND (activity_date - window_start) <= 2 -- window is truly 3 days
GROUP BY activity_date
ORDER BY activity_date;
```

ASCII Visual Diagrams

Gap and Island Detection (P14)

User 1 activity dates: Jan 1, 2, 3, 4, 7 (gap on 5 and 6)

date	ROW_NUMBER	date - row_num	= grp
2024-01-01	1	2024-01-01 - 1	= 2023-12-31 ← group 1
2024-01-02	2	2024-01-02 - 2	= 2023-12-31 ← group 1
2024-01-03	3	2024-01-03 - 3	= 2023-12-31 ← group 1
2024-01-04	4	2024-01-04 - 4	= 2023-12-31 ← group 1
2024-01-07	5	2024-01-07 - 5	= 2024-01-02 ← group 2

Streak 1: Jan 1 → Jan 4 (4 days)

Streak 2: Jan 7 → Jan 7 (1 day)

Running Total Pattern

Region: North

sale_date	amount	running_total
-----------	--------	---------------

2024-01-05	2400	2400
2024-02-01	1200	3600
2024-03-15	1200	4800
2024-04-01	240	5040
2024-05-15	450	5490

Window: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Each row sees itself + all prior rows in the partition

Common Interview Mistakes

1. Using RANGE instead of ROWS for running totals
→ Unexpected dedup behavior with ties
2. Forgetting PARTITION BY → computing global instead of per-group
3. Forgetting LAST_VALUE frame → returns current row, not last
4. Filtering on window function in WHERE (invalid)
→ Wrap in subquery/CTE
5. Using RANK when DENSE_RANK was needed for "Nth value" problems
6. LAG/LEAD missing PARTITION BY → crossing group boundaries
7. Not handling NULL in NULLIF(denominator, 0) for growth rates
8. Missing tie-breaker in ORDER BY → non-deterministic ROW_NUMBER

Interview Strategy Guide

When you see:

Use this:

"Rank by X within group Y"
ORDER BY X)

RANK / DENSE_RANK / ROW_NUMBER OVER (PARTITION BY Y

"Top N per group"

ROW_NUMBER WHERE <= N

"Running total"

SUM(...) OVER (ORDER BY date ROWS UNBOUNDED

PRECEDING ...)

"Previous period comparison"

LAG(col) OVER (PARTITION BY grp ORDER BY date)

"Next period comparison"

LEAD(col) OVER (...)

"First/last value in group"

FIRST_VALUE / LAST_VALUE (remember frame!)

"Consecutive streak / gap/island"

date - ROW_NUMBER() technique

"Nth highest value"

DENSE_RANK WHERE = N

"Percentile / median"

PERCENTILE_CONT(0.5) WITHIN GROUP

"Divide into N equal groups"

NTILE(N)

"Compare to group average"	AVG(...) OVER (PARTITION BY group) alongside row
"Moving average"	AVG(...) OVER (ROWS BETWEEN N PRECEDING AND CURRENT ROW)
"Deduplication"	ROW_NUMBER = 1 (keep latest/first)

Best Practices for Interviews

1. Think aloud — explain the window you're defining before writing it.
2. Write the window function inline first, wrap in CTE/subquery to filter.
3. Always specify the frame clause for FIRST_VALUE, LAST_VALUE, NTH_VALUE.
4. Use the WINDOW clause to avoid repeating identical window definitions.
5. Add a tie-breaker column to ORDER BY for deterministic results.
6. Test edge cases: what happens with NULLs? what if the group has 1 row?

Performance Considerations

```
-- 1. Each distinct OVER() definition adds a Sort + WindowAgg
-- Use named WINDOW to share definitions

-- 2. Index on PARTITION BY + ORDER BY columns:
CREATE INDEX idx_sales_region_date ON sales_iv(region, sale_date);
CREATE INDEX idx_emp_dept_salary ON employees_iv(department, salary DESC);

-- 3. For top-N with index: LATERAL + LIMIT is often faster than ROW_NUMBER subquery
-- For full-partition aggregates: window function is usually faster than LATERAL

-- 4. Avoid computing the same window multiple times:
-- WRONG (computes window twice):
SELECT *, ROW_NUMBER() OVER w, RANK() OVER w
FROM t WINDOW w AS (PARTITION BY dept ORDER BY salary);
-- RIGHT: this is actually fine – PostgreSQL shares the sort for same-window

-- 5. For deduplication, DISTINCT ON is simpler and faster for top-1:
SELECT DISTINCT ON (customer_id) *
FROM orders ORDER BY customer_id, order_date DESC;
```

Interview Questions & Answers

Q1: A table has one row per day per user. How do you find users who were active for 7 consecutive days?

A: Use the gap-and-island technique: subtract ROW_NUMBER() from the date to get a group constant for consecutive runs. Then COUNT(*) in each group and filter WHERE count >= 7:

```
WITH grps AS (
    SELECT user_id, activity_date,
           activity_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
activity_date)::INT AS grp
    FROM user_activity
```

```
)
SELECT user_id FROM grps GROUP BY user_id, grp HAVING COUNT(*) >= 7;
```

Q2: How do you compute a 7-day moving average?

A:

```
AVG(metric) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

Use ROWS (not RANGE) for exact 7-row windows regardless of date gaps.

Q3: How would you find the employee who had the biggest salary jump from their previous position?

A: Use LAG to get the previous salary, compute the difference, then rank by the difference:

```
SELECT * FROM (
  SELECT emp_name, salary,
         salary - LAG(salary) OVER (ORDER BY hire_date) AS salary_jump,
         RANK() OVER (ORDER BY salary - LAG(salary) OVER (ORDER BY hire_date)
                     DESC) AS rnk
  FROM employees_iv
) t WHERE rnk = 1;
```

Q4: How do you handle a running total that resets at each year?

A:

```
SUM(amount) OVER (PARTITION BY EXTRACT(YEAR FROM sale_date) ORDER BY sale_date
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_total
```

Adding the year to PARTITION BY causes the running total to reset each year.

Q5: A transaction table has deposits and withdrawals. How do you compute the running balance?

A:

```
SUM(CASE WHEN type = 'deposit' THEN amount ELSE -amount END)
OVER (PARTITION BY account_id ORDER BY txn_date, txn_id
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_balance
```

Q6: How would you detect if a customer's order value has been declining for 3 consecutive months?

A: Use LAG twice to get the previous and two-previous months, then filter where the current is less than previous AND previous is less than two-previous.

Q7: What is the difference between running total with ROWS and RANGE?

A: ROWS uses physical row offsets — the frame is always exactly N rows. RANGE uses logical value ranges — all rows with the same ORDER BY value are treated as peers and included together. For dates with possible duplicate values, use ROWS for predictable behavior.

Q8: You have sales by day. How do you find the day with the highest sales in each month?

A: Use DENSE_RANK or RANK partitioned by month, ordered by daily sales DESC, then filter WHERE rank = 1.
Alternatively use DISTINCT ON:

```
SELECT DISTINCT ON (DATE_TRUNC('month', sale_date))
    DATE_TRUNC('month', sale_date) AS month, sale_date, SUM(amount) AS daily_revenue
FROM sales_iv
GROUP BY sale_date
ORDER BY DATE_TRUNC('month', sale_date), SUM(amount) DESC;
```

Additional Practice Exercises

Extra 1: Salary Bands Distribution

Show what percentage of employees fall into each \$20K salary band, with running percentage.

```
-- Solution:
WITH banded AS (
    SELECT
        emp_name, salary,
        FLOOR(salary/20000)*20000 AS band
    FROM employees_iv
)
SELECT
    band::TEXT || '-' || (band+19999)::TEXT AS salary_range,
    COUNT(*) AS emp_count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct,
    ROUND(100.0 * SUM(COUNT(*)) OVER (ORDER BY band
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
        / SUM(COUNT(*)) OVER (), 2) AS cumulative_pct
FROM banded
GROUP BY band
ORDER BY band;
```

Advanced Notes

Pattern: Self-Join Replaced by Window Function

Many classic self-join problems that were solved with correlated subqueries can now be solved more efficiently with window functions:

```
-- Old way (correlated subquery):
SELECT e1.emp_name, e1.salary FROM employees e1
WHERE e1.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.dept = e1.dept);

-- New way (window function):
SELECT emp_name, salary FROM (
    SELECT emp_name, salary,
        AVG(salary) OVER (PARTITION BY department) AS dept_avg
    FROM employees_iv
```

```
) t WHERE salary > dept_avg;  
-- One table scan instead of N+1
```

Cross-references

- **01_window_functions_intro.md** — OVER, PARTITION BY, frame clauses
- **02_ranking_functions.md** — ROW_NUMBER, RANK, DENSE_RANK, NTILE
- **03_analytical_functions.md** — LAG, LEAD, FIRST_VALUE, LAST_VALUE
- **04_ctes.md** — CTEs used to stage window function results
- **07_lateral_joins.md** — LATERAL as alternative to window function patterns
- **19_Interview_Questions/** — More interview preparation material